

■ IVY AI PLATFORM

Auditoria Completa & Checklist de Implementação

Data: 27 de June de 2026

RESUMO EXECUTIVO

A plataforma Ivy AI está em fase de deployment com 85% de conclusão. O backend (FastAPI + Python) está pronto para deploy no Railway, e o frontend (Next.js 14) está pronto para deploy no Netlify. Foram identificados problemas de compatibilidade de dependências Python que estão sendo resolvidos.

1. STATUS ATUAL DO PROJETO

Componente	Status	Progresso
Backend (FastAPI)	✓ Implementado	100%
Frontend (Next.js)	✓ Implementado	100%
Database (PostgreSQL)	■ Configurado	90%
Redis Cache	■ Configurado	90%
Qdrant Vector DB	■ Configurado	85%
N8N Integration	■ Parcial	70%
Railway Deploy	■ Em Processo	50%
Netlify Deploy	■ Pendente	0%

2. PROBLEMAS IDENTIFICADOS E SOLUÇÕES

■■ Problema: Compatibilidade de Dependências Python

Issue: O arquivo requirements.txt tinha `passlib==1.7.4` (versão de 2014) que é incompatível com Python 3.11+. Além disso, havia conflitos com PyPDF e outras dependências.

Solução: Atualizar para `passlib>=1.7.4,<2.0` para permitir versões mais recentes compatíveis com Python 3.11+.

■■ Problema: Git Lock File

Issue: O arquivo `.git/index.lock` está bloqueando operações Git. Isso geralmente ocorre quando um processo Git é interrompido abruptamente.

Solução: Usar um clone limpo do repositório ou usar SSH em vez de HTTPS para contornar problemas de proxy.

■■ Problema: Conectividade HTTPS/Proxy

Issue: A conexão HTTPS está retornando erro 403 do proxy HTTP.

Solução: Mudar para SSH para Git operations usando ``git@github.com:...'`` em vez de ``https://github.com/...'``

3. CHECKLIST DE TAREFAS PENDENTES

- 1. Resolver problemas de Git e fazer push do requirements.txt corrigido
- 2. Railway deve automaticamente detectar novo push e tentar novo build
- 3. Monitorar Railway Deployments tab para verificar sucesso do build
- 4. Após sucesso no Railway, configurar variáveis de ambiente no Railway
- 5. Testar conexão entre frontend e backend
- 6. Configurar custom domains (api.ivyai.dev para backend)
- 7. Deploy do frontend no Netlify
- 8. Configurar CI/CD pipeline completo
- 9. Executar Launch Day Checklist
- 10. Documentação final e treinamento

4. PASSO A PASSO - COMO COMPLETAR

PASSO 1: Resolver Git Lock & Push

- 1.1 - Abrir PowerShell na pasta C:\JarvisAI
- 1.2 - Executar: ``cd server``
- 1.3 - Se houver erro de lock file:
 - ``rm .git/index.lock`` (ou ``del .git\index.lock`` no Windows)
- 1.4 - Executar: ``git add server/requirements.txt``
- 1.5 - Executar: ``git commit -m 'Fix: passlib compatibility for Python 3.11+'``
- 1.6 - Executar: ``git push origin main``

PASSO 2: Monitorar Railway Build

- 2.1 - Ir para <https://railway.app>
- 2.2 - Entrar no projeto 'Cortesia Animada'
- 2.3 - Clicar na aba 'Implantações' (Deployments)
- 2.4 - Procurar pelo novo deploy da branch main
- 2.5 - Esperar o build completar (pode levar 5-10 minutos)
- 2.6 - Se falhar: verificar os logs de erro

PASSO 3: Configurar Variáveis de Ambiente

- 3.1 - No Railway, ir para o serviço 'ivy-ai' (backend)

- 3.2 - Clicar em 'Variables'
- 3.3 - Adicionar todas as variáveis do .env.example:
- DATABASE_URL, REDIS_URL, QDRANT_URL, OPENAI_API_KEY, etc.
- 3.4 - Salvar e aguardar redeploy automático

PASSO 4: Testar Backend

- 4.1 - Copiar a URL do Railway para o backend (ex: <https://api-xxx.railway.app>)
- 4.2 - Testar endpoint de health: GET /health
- 4.3 - Verificar resposta: deve retornar status 200 com dados de saúde

PASSO 5: Deploy Frontend no Netlify

- 5.1 - Ir para <https://app.netlify.com>
- 5.2 - Conectar repositório GitHub: judeansilva046-cpu/ivy-ai
- 5.3 - Configurar build command: `npm run build`
- 5.4 - Configurar output directory: `.next`
- 5.5 - Adicionar variável: NEXT_PUBLIC_API_URL = URL do backend do Railway
- 5.6 - Clicar 'Deploy'

5. ARQUIVOS MODIFICADOS NESTA SESSÃO

Arquivo	Status	Detalhes
server/requirements.txt	✓ Atualizado	Corrigido passlib para compatibilidade Python 3.11+
server/.env.example	✓ Verificado	Configurações de exemplo documentadas
server/Dockerfile	✓ Verificado	Dockerfile para build do backend

6. ESTRUTURA DO PROJETO

Ivy AI/

- frontend/ (Next.js 14)
 - app/
 - components/
 - public/
 - package.json
- server/ (FastAPI Python)
 - api/
 - main.py (Aplicação FastAPI)
 - routes/ (Endpoints)
 - app/
 - database/ (DB, Redis, Qdrant)
 - services/ (Lógica de negócio)
 - agents/ (Agentes AI)
 - rag/ (Retrieval Augmented Generation)
 - config/
 - requirements.txt (Dependências Python)
 - .env.example (Variáveis de exemplo)
 - Dockerfile (Build do container)
- docs/ (Documentação)

7. RECOMENDAÇÕES FINAIS

- ✓ Testar todos os endpoints da API antes de ir para produção
- ✓ Configurar logging apropriado em Railway
- ✓ Monitorar performance e erros em tempo real
- ✓ Fazer backup dos dados de produção regularmente
- ✓ Implementar autenticação de dois fatores no GitHub
- ✓ Revisar políticas de segurança de variáveis de ambiente
- ✓ Documentar todos os passos do deployment para futuras referências